

Lab Assignment V

Maximum marks: 20

13/02/2025

- Use pgAdmin, and write all queries in the same file with question numbers.
- Once you complete doing a question, show the query and result to the TA and give an explanation if any. You needn't write explanations if you have explained it to the TA.
- Write a query, execute and paste screenshots, and make a pdf. Add question numbers before the answers. You need to show a screenshot to demonstrate the correctness of your query. The same screenshot should have the query and its output. If you have made some changes that are not visible in the direct output, display the corresponding tables again and add their screenshots.
- Submit one .pdf file containing all answers named <roll no>_assignment5.pdf.
- Write a small justification for your solution, if required.
- Submit the query file from pgAdmin named <roll no>_assignment5.sql along with the pdf. It should contain all the queries you have used. Queries should not be commented.
- Complete the exercise before 4:45 PM and submit the .pdf and .sql in Moodle as two files.

Q1.

Write a function that takes store_id, start_date, end_date and returns:

- total rentals
- total revenue **(2 marks)**

Raise exception if -

- i) store does not exist **(0.5 marks)**
- ii) start_date > end_date. **(0.5 marks)**

Use input as (1, '2005-01-01', '2005-12-31')

Q2.

Write a **procedure** that takes:

- `category_id`
- `percentage_increase`

The procedure should:

- Increase the `rental_rate` of films belonging to the given category
- Apply the increase **only to films whose rental_rate is less than 3.00**

Demonstrate the procedure using:

category_id = 3

percentage_increase = 10

Show the results before and after calling the procedure.

(0.5 +2+0.5 marks)

Q3.

Write a function that takes `customer_id` and returns:

- total rentals
- total payments
- average days rented

Show output for customer id=1

If customer not found → raise exception

(2.5 + 0.5 marks)

Q4.

Write a function that takes:

- film_id
- store_id

Return:

- total copies
- rented copies (return_date IS NULL)
- available copies

Verify for film_id=1 and store_id=1

(3 marks)

Q5.

Write a function that takes:

- minimum_total_payment

Return a table of customers:

- customer_id
- first_name
- last_name
- total_payment

Include only customers whose total_payment > given value. **Verify for input 150.**

If no customer found → raise exception.

(2.5 + 0.5 marks)

Q6.

Write a procedure that takes:

`minimum_payment` NUMERIC

The procedure should:

1. Add a new column *loyalty_status* VARCHAR(20) to the customer table (if it does not already exist).
2. Update customers as follows:

Condition	loyalty_status
-----------	----------------

<code>total_payment > minimum_payment</code>	'Premium'
---	-----------

<code>total_payment <= minimum_payment</code>	'Regular'
--	-----------

Only update customers who have made at least one payment.

Demonstrate the procedure with:

`minimum_payment = 150`

Show results **before and after** calling the procedure.

(5 marks)

